

Trabalho Individual II - SCOM

Este trabalho é a continuação do Trabalho Individual I. O estudante deverá evoluir o sistema anteriormente desenvolvido, adicionando um backend funcional, persistência de dados, autenticação, controle de acesso e mecanismos de qualidade de software.

O tema do projeto permanece livre. O backend deverá ser coerente com o domínio escolhido no Trabalho 1.

Objetivos de aprendizagem

Ao final do trabalho, o estudante deverá demonstrar capacidade de:

- projetar uma arquitetura cliente-servidor;
- implementar APIs seguras;
- persistir dados em banco relacional;
- autenticar usuários;
- aplicar controle de acesso;
- validar e sanitizar entradas;
- testar funcionalidades e segurança;
- medir desempenho;
- documentar APIs;
- utilizar Git de forma profissional;
- utilizar IA generativa de forma crítica e rastreável.

Serão permitidas quaisquer tecnologias escolhidas pelo aluno, mas sugere-se PHP ou JavaScript everywhere para o backend, PostgreSQL ou MySQL/MariaDB para construção do banco de dados.

Requisitos obrigatórios

1. Arquitetura

O projeto deverá apresentar separação clara entre:

- interface;
- lógica de aplicação;
- acesso a dados.

É obrigatório documentar a arquitetura adotada (MVC, camadas, hexagonal simplificada, etc.).

2. API e persistência

Implementar pelo menos:

- uma entidade principal persistente;
- operações de criação, leitura, atualização e exclusão compatíveis com o domínio;
- acesso ao banco por consultas parametrizadas ou ORM;
- constraints adequadas (PK, FK, UNIQUE, NOT NULL).

É obrigatório entregar o script de criação do banco.

3. Autenticação

Implementar autenticação por:

- sessão segura ou
- JWT.

Exigências mínimas:

- armazenamento de senha com hash seguro (`password_hash`/bcrypt/Argon2);
- logout funcional;
- proteção de rotas autenticadas.

4. Controle de acesso

Implementar pelo menos dois perfis de acesso coerentes com o domínio.

Exemplos:

- administrador/usuário;
- professor/aluno;
- operador/visitante.

O relatório deverá descrever a matriz de permissões.

5. Validação e sanitização

Toda entrada externa deverá ser validada no backend.

Itens obrigatórios:

- validação de tipos;
- validação de tamanho;
- sanitização quando aplicável;
- mensagens de erro estruturadas.

6. Segurança mínima obrigatória

O sistema deverá demonstrar proteção contra:

- SQL Injection;
- XSS refletido;
- CSRF (quando utilizar formulários com sessão);

- acesso não autorizado a rotas protegidas.

O relatório deverá apresentar evidências dos testes realizados.

7. Tratamento de erros

Implementar respostas padronizadas contendo:

- código de status HTTP;
- mensagem legível;
- detalhes técnicos apenas em modo de desenvolvimento.

8. Documentação da API

Documentar todos os endpoints em formato Markdown ou OpenAPI simplificado contendo:

- método HTTP;
- URL;
- parâmetros;
- exemplo de requisição;
- exemplo de resposta;
- códigos de status possíveis.

9. Testes automatizados

Implementar testes para a lógica de negócio.

Mínimo exigido: 5 testes automatizados executáveis.

Os testes devem cobrir casos válidos e inválidos.

10. Observabilidade básica

Implementar registro de logs de aplicação contendo pelo menos:

- login bem-sucedido;
- falha de autenticação;
- criação de entidade;
- erro interno.

Os logs devem possuir timestamp.

Métricas objetivas de avaliação

A. Arquitetura e organização – 15 pts

1 ponto para cada item atendido (máx. 15):

- separação frontend/backend;
- diretórios organizados;
- camada de acesso a dados isolada;
- configuração separada do código;
- variáveis sensíveis fora do código-fonte;
- script SQL fornecido;
- README de execução;
- documentação da arquitetura;
- nomenclatura consistente;
- ausência de arquivos temporários;
- ausência de código morto evidente;
- uso consistente de namespaces ou módulos;
- tratamento centralizado de erros;
- reutilização de componentes;
- estrutura compatível com o padrão descrito.

B. Banco de dados – 15 pts

Pontuação:

- PK em todas as tabelas (3);
- FKs corretas (3);
- constraints adequadas (3);
- tipos apropriados (2);
- índices relevantes (2);
- script executável sem erros (2).

C. Segurança – 25 pts

C1. Proteções obrigatórias (15 pts)

- prepared statements/ORM seguro (5);
- escaping de saída HTML (3);
- proteção CSRF quando aplicável (3);
- controle de acesso efetivo (4).

C2. Testes de segurança documentados (10 pts)

2 pontos por teste comprovado (máx. 5 testes).

Exemplos aceitos:

- tentativa de SQL Injection;
- tentativa de XSS;
- acesso sem login;
- acesso com perfil insuficiente;
- reutilização de token/sessão inválida.

D. Autenticação e autorização – 15 pts

- hash seguro de senhas (4);
- login funcional (3);
- logout funcional (2);
- proteção de rotas (3);
- diferenciação de perfis funcional (3).

E. Qualidade da API – 10 pts

- uso correto de métodos HTTP (2);
- códigos de status adequados (2);
- respostas consistentes em JSON quando aplicável (2);
- documentação completa (4).

F. Testes automatizados – 10 pts

- ≥5 testes executáveis (5);
- cobertura de casos inválidos (3);
- evidência de execução (2).

G. Git e processo de desenvolvimento – 10 pts

- organizado, com commits significativos e logs descritivos (3);
- mensagens padronizadas (1);
- evolução incremental identificável (2);
- uso de branches ou tags para marcos do projeto (4).

Boas práticas opcionais (+5 pts cada, sendo a nota final limitada a 100 pontos):

- `.env` para configuração;
- migrações de banco;
- paginação de resultados;
- rate limiting simples;
- testes de integração automatizados.

Relatório técnico obrigatório

Entregar relatório em PDF contendo:

1. Introdução;
2. Evolução em relação ao Trabalho 1;
3. Requisitos funcionais;
4. Arquitetura do sistema;
5. Modelo de dados;
6. Estratégia de autenticação;
7. Estratégia de segurança;
8. Endpoints implementados;

9. Estratégia de testes;
10. Resultados dos testes de segurança;
11. Logs e observabilidade;
12. Desempenho e limitações;
13. Uso de IA generativa;
14. Conclusões.

Evidências obrigatórias

Anexar ao relatório:

- exportação do banco ou script SQL;
- documentação da API;
- resultado dos testes automatizados;
- capturas dos testes de segurança;
- amostra dos logs;
- histórico Git (`git log --oneline --graph`);
- checklist de requisitos preenchido.

Penalidades

- ausência de relatório: -20 pts;
- ausência de vídeo: -10 pts;
- ausência de repositório Git: -15 pts;
- uso de IA não declarado: até nota zero;
- credenciais hardcoded no repositório: -10 pts;
- sistema que não executa localmente: -15 pts.

Entrega

Enviar:

1. código-fonte completo;
2. script SQL;
3. relatório PDF;
4. link do repositório Git.